海康摄像头气密检测任务实现方案

基于 **RoboOS (RoboBrain任务分解) + VLA执行原子任务** + Fourier GR2 机器人的实现方案

一、任务概述

1.1 任务背景

本任务为海康摄像头产品的气密检测流程自动化，使用傅利叶 GR2 人形机器人（双臂7DOF + 五指灵巧手）完成全部9个操作步骤。

1.2 核心设计理念

采用 **"RoboBrain任务分解 + VLA执行原子任务"** 的双层架构：

层级	模型	职责
任务分解层	RoboBrain 2.0 (Master)	理解自然语言指令，分解为原子任务序列
任务执行层	VLA模型 (Slaver)	执行具体原子技能，输出机器人控制指令

1.3 任务流程

步骤	任务名称	描述
1	取摄像头产品	左手从周转盒内双指拿取摄像头
2	摘镜头帽+检查	右手摘掉镜头帽并放置，目测检查镜头
3	扫描二维码	双手转动摄像头，将二维码对准扫码器
4	放置工装	镜头朝上水平放入气密检测工装
5	启动检测	双手同时按下两个绿色启动按钮
6	取出+检查	检测完成后取出摄像头，目测检查
7	盖镜头帽	左手取镜头帽盖到摄像头上
8	放入周转盒	按顺序整齐放入周转盒
9	查看结果	读取屏幕上的气密检测结果

二、系统架构

2.1 整体架构：RoboBrain分解 + VLA直接输出控制参数

```
┌──────────────────────────────────────────────────────────────────────┐
│                          用户自然语言指令                              │
│                    "执行海康摄像头气密检测流程"                         │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼

┌──────────────────────────────────────────────────────────────────────┐
│                     RoboOS Master Agent                              │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │              RoboBrain 2.0 (任务分解模型)                     │  │
│  │                                                              │  │
│  │  输入: "执行海康摄像头气密检测流程"                            │  │
│  │                                                              │  │
│  │  输出: [                                                     │  │
│  │    {"subtask": "从周转盒取摄像头", "robot": "gr2"},          │  │
│  │    {"subtask": "摘除镜头帽并检查", "robot": "gr2"},          │  │
│  │    {"subtask": "扫描产品二维码", "robot": "gr2"},            │  │
│  │    ...                                                       │  │
│  │  ]                                                           │  │
│  └──────────────────────────────────────────────────────────────┘    │
│                         GlobalTaskPlanner                            |
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │ Redis 发布子任务
                                ▼

┌──────────────────────────────────────────────────────────────────────┐
│                     RoboOS Slaver Agent                              │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                ToolCallingAgent (ReAct)                      │  │
│  │                                                              │  │
│  │  接收子任务: "从周转盒取摄像头"                                │  |
│  │                                                              │  │
│  │  思考 → 选择VLA技能 → 执行 → 观察 → 循环                      |  |
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │ MCP Tool Call
                                ▼

┌──────────────────────────────────────────────────────────────────────┐
│                 VLA原子技能 (端到端动作生成)                           |
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                      VLA Model                               |  |
│  │          (DexGraspVLA / OpenVLA / RDT / π0)                  |  |
│  │                                                              |  |
│  │  输入:                                                       |  |
│  │  • 第三视角RGB图像 + 目标Mask                                |  |
│  │  • 腕部相机RGB图像                                           |  |
│  │  • 当前机器人状态 (13-DoF)                                   |  |
│  │  • 语言指令 (可选)                                           |  |
│  │                                                              |  |
```

```
│  │  输出:                                                    │  │
│  │  • 动作序列 action: (T, 13) ← 直接输出关节角度!            │  │
│  │    - 手臂7-DoF: [肩俯仰,肩侧摆,肩偏航,肘俯仰,腕偏航,      │  │
│  │                  腕俯仰,腕侧摆]                            │  │
│  │    - 灵巧手6-DoF: [拇指1,拇指2,食指,中指,无名指,小指]      │  │
│  └──────────────────────────────────────────────────────────┘  │
│                              │                                  │
│                              ▼ 直接输出关节角度指令              │
└──────────────────────────────┴──────────────────────────────────┘
                              │ Aurora Client API (无需坐标转换)
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                      Fourier GR2 机器人                          │
│                  (双臂7DOF + 灵巧手6DOF)                          │
│                                                                  │
│   client.set_joint_positions({                                   │
│       "right_manipulator": action[0:7],   # 手臂7关节            │
│       "right_hand": action[7:13]          # 灵巧手6关节          │
│   })                                                             │
└──────────────────────────────────────────────────────────────────┘
```

2.2 VLA端到端控制的优势

特性	传统方案 (感知→规划→控制)	VLA端到端方案
流程	像素→世界坐标→IK→关节角度	像素 → 关节角度 (端到端)
中间表示	bbox, 3D点云, 轨迹点	无，直接输出动作
标定依赖	需要手眼标定、相机内参	无需标定，从数据学习
泛化能力	依赖标定精度	从大规模数据学习泛化
误差累积	多阶段误差累积	端到端，无累积
实时性	多次推理	单次推理输出动作序列

2.3 数据流详解

```
用户指令: "执行海康摄像头气密检测"
        │
        ▼ RoboBrain理解+分解
┌──────────────────────────────────────┐
│ 子任务1: "从周转盒取摄像头"            │
│ 子任务2: "摘除镜头帽"                  │
│ 子任务3: "检查镜头外观"                │
│ ...                                  │
└──────────────────────────────────────┘
        │
        ▼ Slaver接收子任务1, 调用VLA技能
┌──────────────────────────────────────────────────────────────────┐
│                     VLA技能: vla_pick_object                     │
```

```
│                                                                    │
│  输入:                                                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐  │
│  │ 第三视角图像 │  │ 腕部相机图像│  │ 当前状态 (13-DoF)       │  │
│  │ + 目标Mask   │  │              │  │ [0.1,0.2,0.0,-0.5,...]  │  │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘  │
│                              │                                      │
│                              ▼                                      │
│  ┌──────────────────────────────────────────────────────────────┐  │
│  │                   VLA Diffusion Policy                       │  │
│  │                                                              │  │
│  │   扩散模型去噪 → 生成动作序列 (T步, 每步13-DoF)            │  │
│  └──────────────────────────────────────────────────────────────┘  │
│                              │                                      │
│                              ▼                                      │
│  输出: action_sequence (T, 13)                                     │
│  ┌──────────────────────────────────────────────────────────────┐  │
│  │ Step 0: [0.12, 0.25, 0.01, -0.48, 0.0, 0.1, 0.0,            │  │
│  │          0.0, 0.0, 0.0, 0.0, 0.0, 0.0]  # 张开手             │  │
│  │ Step 1: [0.15, 0.30, 0.02, -0.45, 0.0, 0.1, 0.0, ...]       │  │
│  │ Step 2: [0.18, 0.35, 0.03, -0.42, 0.0, 0.1, 0.0, ...]       │  │
│  │ ...                                                          │  │
│  │ Step T: [0.25, 0.50, 0.05, -0.30, 0.0, 0.1, 0.0,            │  │
│  │          1.2, 1.2, 1.0, 1.0, 0.8, 0.0]  # 抓取完成           │  │
│  └──────────────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────────────┘
        │
        ▼ 直接发送到机器人执行

┌──────────────────────────────────────────┐
│ for action in action_sequence:           │
│     client.set_joint_positions({         │
│         "right_manipulator": action[:7],│
│         "right_hand": action[7:13]       │
│     })                                   │
│     sleep(0.01)  # 100Hz控制             │
└──────────────────────────────────────────┘
        │
        ▼ 子任务1完成，执行子任务2...
```

三、RoboBrain任务分解层

3.1 Master Agent 任务分解流程

RoboOS Master 使用 RoboBrain 2.0 模型将用户的自然语言指令分解为可执行的子任务序列。

输入示例：

```
"执行海康摄像头气密检测流程"
```

RoboBrain分解输出：

```
{
  "task": "海康摄像头气密检测",
  "subtasks": [
    {
      "step": 1,
      "description": "从周转盒中取出一个摄像头",
      "robot": "gr2",
      "preconditions": [],
      "expected_state": {"left_hand": "holding_camera"}
    },
    {
      "step": 2,
      "description": "摘除摄像头上的镜头帽并放置到指定位置",
      "robot": "gr2",
      "preconditions": ["left_hand_holding_camera"],
      "expected_state": {"lens_cap": "placed", "lens": "exposed"}
    },
    {
      "step": 3,
      "description": "目视检查镜头外观是否有损伤",
      "robot": "gr2",
      "preconditions": ["lens_exposed"],
      "expected_state": {"inspection": "passed"}
    },
    {
      "step": 4,
      "description": "转动摄像头使二维码朝向扫码器并完成扫码",
      "robot": "gr2",
      "preconditions": ["inspection_passed"],
      "expected_state": {"qrcode": "scanned"}
    },
    {
      "step": 5,
      "description": "将摄像头镜头朝上放入气密检测工装槽位",
      "robot": "gr2",
      "preconditions": ["qrcode_scanned"],
      "expected_state": {"camera": "in_fixture"}
    },
    {
      "step": 6,
      "description": "双手同时按下两个绿色启动按钮",
      "robot": "gr2",
      "preconditions": ["camera_in_fixture"],
      "expected_state": {"detection": "started"}
    },
    {
      "step": 7,
      "description": "等待气密检测完成后从工装中取出摄像头",
      "robot": "gr2",
      "preconditions": ["detection_completed"],
      "expected_state": {"right_hand": "holding_camera"}
    },
    {
```

```
      "step": 8,
      "description": "检查取出后的摄像头外观",
      "robot": "gr2",
      "preconditions": ["right_hand_holding_camera"],
      "expected_state": {"post_inspection": "passed"}
    },
    {
      "step": 9,
      "description": "将镜头帽盖回摄像头上",
      "robot": "gr2",
      "preconditions": ["post_inspection_passed"],
      "expected_state": {"lens_cap": "on"}
    },
    {
      "step": 10,
      "description": "将摄像头镜头朝下放入周转盒",
      "robot": "gr2",
      "preconditions": ["lens_cap_on"],
      "expected_state": {"camera": "in_box"}
    },
    {
      "step": 11,
      "description": "查看屏幕上的气密检测结果",
      "robot": "gr2",
      "preconditions": ["camera_in_box"],
      "expected_state": {"result": "recorded"}
    }
  ]
}
```

3.2 任务分解Prompt模板

```
MASTER_PLANNING_PROMPT = """
你是一个机器人任务规划专家。请将用户的任务分解为可执行的子任务序列。

可用机器人: {robot_name_list}
机器人能力: {robot_tools_info}
当前场景: {scene_info}

用户任务: {task}

请输出JSON格式的子任务列表，每个子任务包含:
- step: 步骤编号
- description: 任务描述（具体、可执行）
- robot: 执行机器人
- preconditions: 前置条件
- expected_state: 预期状态变化

注意:
1. 子任务应该是原子级别，可以直接被VLA模型执行
2. 考虑双臂协作和手部状态
3. 包含必要的检查和验证步骤
```

```
"""
```

3.3 场景状态记录

RoboOS通过SceneMemory记录任务执行过程中的状态变化：

```
# 场景状态示例
scene_state = {
    "objects": {
        "camera_01": {
            "location": "left_hand",
            "state": "grasped",
            "lens_cap": "removed"
        },
        "lens_cap_01": {
            "location": "table_left_area",
            "state": "placed"
        }
    },
    "robot_state": {
        "left_hand": "holding_camera",
        "right_hand": "empty"
    },
    "task_progress": {
        "current_step": 3,
        "completed_steps": [1, 2]
    }
}
```

四、VLA原子技能执行层

VLA模型采用端到端方式，直接从视觉输入生成机器人关节角度指令，无需中间的坐标转换。

4.1 VLA原子技能列表

每个原子技能对应一个训练好的VLA策略，直接输出13-DoF动作序列。

技能代码	技能名称	输入	输出	说明
VLA_PICK	抓取物体	RGB+Mask+State	action(T,13)	从指定位置抓取目标物体
VLA_PLACE	放置物体	RGB+Mask+State	action(T,13)	将物体放置到指定位置
VLA_PULL	拔出操作	RGB+Mask+State	action(T,13)	拔出镜头帽等插入物
VLA_INSERT	插入操作	RGB+Mask+State	action(T,13)	插入镜头帽等
VLA_PRESS	按压操作	RGB+Mask+State	action(T,13)	按压按钮
VLA_ROTATE	旋转物体	RGB+Mask+State	action(T,13)	双手协同旋转物体
VLA_HANDOVER	双手传递	RGB+Mask+State	action(T,13)	左右手物体传递

技能代码	技能名称	输入	输出	说明
VLA_INSPECT	视觉检测	RGB	缺陷描述	外观缺陷检测(VLM)
VLA_READ	屏幕读取	RGB	文本内容	OCR读取结果(VLM)

4.2 VLA模型输入输出规范

```
# VLA模型输入
vla_input = {
    "rgb_third_view": np.ndarray,      # (H, W, 3) 第三视角RGB图像
    "mask": np.ndarray,                 # (H, W, 1) 目标物体二值Mask
    "rgb_wrist": np.ndarray,            # (H, W, 3) 腕部相机RGB图像
    "robot_state": np.ndarray,          # (13,) 当前机器人状态
    "language_instruction": str         # 可选, 语言指令
}

# VLA模型输出 - 直接是关节角度!
vla_output = {
    "action_sequence": np.ndarray,      # (T, 13) 动作序列
    # action[0:7]  = 手臂关节角度 [肩俯仰,肩侧摆,肩偏航,肘俯仰,腕偏航,腕俯仰,腕侧摆]
    # action[7:13] = 灵巧手关节角度 [拇指1,拇指2,食指,中指,无名指,小指]
}
```

4.3 VLA技能执行流程

```
┌──────────────────────────────────────────────────────────────────────┐
│                        VLA原子技能执行                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  1. 获取当前观测                                                      │
│     ┌──────────────┐  ┌──────────────┐  ┌──────────────┐            │
│     │ 第三视角相机 │  │ 腕部相机     │  │ 机器人状态   │            │
│     │ RGB (H,W,3) │  │ RGB (H,W,3) │  │ State (13,) │            │
│     └──────┬───────┘  └──────┬───────┘  └──────┬───────┘            │
│            │                 │                 │                     │
│  2. 目标分割 (SAM/Cutie)    │                 │                     │
│     ┌──────▼───────┐         │                 │                     │
│     │ 目标Mask     │         │                 │                     │
│     │ (H,W,1)      │         │                 │                     │
│     └──────┬───────┘         │                 │                     │
│            │                 │                 │                     │
│  3. VLA推理 ◀────────────────┴─────────────────┘                     │
│     ┌──────▼──────────────────────────────────────────────┐          │
│     │              VLA Diffusion Policy                    │          │
│     │                                                      │          │
│     │   • 图像编码 (ViT)                                   │          │
│     │   • 状态编码                                         │          │
│     │   • 扩散去噪 (DDPM/DDIM)                             │          │
│     │   • 输出: action_sequence (T, 13)                    │          │
│     └──────┬──────────────────────────────────────────────┘          │
│            │                                                          │
```

```
│            │                                                    │
│  4. 直接执行动作序列                                              │
│     ┌──────▼──────────────────────────────────────────────┐    │
│     │  for t in range(T):                                  │    │
│     │      action = action_sequence[t]                     │    │
│     │      client.set_joint_positions({                    │    │
│     │          "right_manipulator": action[0:7],           │    │
│     │          "right_hand": action[7:13]                  │    │
│     │      })                                              │    │
│     │      sleep(0.01)  # 100Hz                            │    │
│     └──────────────────────────────────────────────────────┘    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

4.4 GR2机器人13-DoF动作空间

```python
# Fourier GR2 单臂+灵巧手 13-DoF 动作空间定义
ACTION_SPACE = {
    # 手臂 7-DoF (单位: 弧度)
    "shoulder_pitch": {"index": 0, "range": [-2.97, 2.97]},   # 肩俯仰
    "shoulder_roll":  {"index": 1, "range": [-0.52, 2.79]},   # 肩侧摆
    "shoulder_yaw":   {"index": 2, "range": [-1.83, 1.83]},   # 肩偏航
    "elbow_pitch":    {"index": 3, "range": [-1.53, 0.48]},   # 肘俯仰
    "wrist_yaw":      {"index": 4, "range": [-1.83, 1.83]},   # 腕偏航
    "wrist_pitch":    {"index": 5, "range": [-0.61, 0.61]},   # 腕俯仰
    "wrist_roll":     {"index": 6, "range": [-0.96, 0.96]},   # 腕侧摆

    # 灵巧手 6-DoF (单位: 弧度)
    "thumb_1":        {"index": 7,  "range": [0.0, 1.57]},    # 拇指关节1
    "thumb_2":        {"index": 8,  "range": [0.0, 1.57]},    # 拇指关节2
    "index_finger":   {"index": 9,  "range": [0.0, 1.57]},    # 食指
    "middle_finger":  {"index": 10, "range": [0.0, 1.57]},    # 中指
    "ring_finger":    {"index": 11, "range": [0.0, 1.57]},    # 无名指
    "pinky_finger":   {"index": 12, "range": [0.0, 1.57]},    # 小指
}

# 双臂操作时使用26-DoF
BIMANUAL_ACTION_SPACE = {
    "left_arm":   [0:7],    # 左臂 7-DoF
    "left_hand":  [7:13],   # 左手 6-DoF
    "right_arm":  [13:20],  # 右臂 7-DoF
    "right_hand": [20:26],  # 右手 6-DoF
}
```

五、各步骤VLA技能执行

每个步骤由RoboBrain分解为子任务，然后调用对应的VLA原子技能，**VLA直接输出13-DoF/26-DoF关节角度动作序列**。

5.1 第一步：取摄像头产品

RoboBrain分解的子任务： `"从周转盒取摄像头"`

VLA技能： `VLA_PICK` → 直接输出 `action(T, 13)`

```
async def step1_pick_camera():
    """VLA直接输出关节角度，无需坐标转换"""

    # 1. 获取观测
    rgb_third = capture_third_view_camera()      # (480, 640, 3)
    rgb_wrist = capture_wrist_camera()           # (480, 640, 3)
    robot_state = get_left_arm_state()           # (13,)

    # 2. SAM分割目标
    mask = sam_segment(rgb_third, prompt="摄像头")

    # 3. VLA推理 - 直接输出关节角度序列！
    action_sequence = vla_model.predict(
        rgb_third_view=rgb_third,
        mask=mask,
        rgb_wrist=rgb_wrist,
        robot_state=robot_state,
        skill="VLA_PICK"
    )  # 输出: (T, 13) 直接是关节角度

    # 4. 直接执行，无需IK/坐标转换
    for action in action_sequence:
        client.set_joint_positions({
            "left_manipulator": action[0:7].tolist(),
            "left_hand": action[7:13].tolist()
        })
        await asyncio.sleep(0.01)  # 100Hz

    return {"success": True}
```

5.2 第二步：摘镜头帽 + 外观检查

VLA技能： `VLA_PULL` (26-DoF双臂) + `VLA_PLACE` + `VLM_INSPECT`

```
async def step2_remove_cap_and_inspect():
    # VLA_PULL: 双臂协同拔出镜头帽
    action_seq = vla_model.predict(skill="VLA_PULL", ...)  # (T, 26)
    for action in action_seq:
        client.set_joint_positions({
            "left_manipulator": action[0:7], "left_hand": action[7:13],
            "right_manipulator": action[13:20], "right_hand": action[20:26]
        })

    # VLA_PLACE: 放置镜头帽
    action_seq = vla_model.predict(skill="VLA_PLACE", ...)  # (T, 13)
```

```
    execute_actions(action_seq, arm="right")

    # VLM检测外观 (非动作任务)
    result = vlm_model.inference(image, prompt="检查镜头缺陷")
    return {"inspection": result}
```

5.3 第三步：扫描二维码

VLA技能：`VLA_ROTATE` (26-DoF双臂协同旋转)

```
async def step3_scan_qrcode():
    action_seq = vla_model.predict(
        skill="VLA_ROTATE",
        language_instruction="将二维码转向扫码器"
    )  # (T, 26) 双臂动作

    for action in action_seq:
        client.set_joint_positions({...})  # 双臂26-DoF

    await wait_for_scanner_response(timeout=5.0)
```

5.4 第四步：放入工装

VLA技能：`VLA_PLACE` → 直接输出精确放置动作

```
async def step4_place_fixture():
    mask = sam_segment(rgb, prompt="工装槽位")
    action_seq = vla_model.predict(
        skill="VLA_PLACE",
        language_instruction="镜头朝上水平放入槽位"
    )
    execute_actions(action_seq, arm="left")
```

5.5 第五步：双手按按钮

VLA技能：`VLA_PRESS` (26-DoF，VLA已学习双手同步)

```python
async def step5_press_buttons():
    mask = sam_segment(rgb, prompt="两个绿色按钮")
    action_seq = vla_model.predict(skill="VLA_PRESS")  # (T, 26)

    # VLA输出的动作天然同步，无需额外处理
    for action in action_seq:
        client.set_joint_positions({
            "left_manipulator": action[0:7], "left_hand": action[7:13],
            "right_manipulator": action[13:20], "right_hand": action[20:26]
        })

    await wait_for_event("detection_complete", timeout=60)
```

5.6 第六步：取出 + 检查

VLA技能：`VLA_PICK` + `VLM_INSPECT`

5.7 第七步：盖镜头帽 (高精度)

VLA技能：`VLA_PICK` + `VLA_INSERT`

VLA从大量演示数据学习精确插入，直接输出±0.3mm精度的动作。

```python
async def step7_replace_cap():
    # 左手抓取镜头帽
    action_seq = vla_model.predict(skill="VLA_PICK", ...)
    execute_actions(action_seq, arm="left")

    # VLA_INSERT: 精确插入，从数据学习精度
    action_seq = vla_model.predict(
        skill="VLA_INSERT",
        language_instruction="将镜头帽盖到镜头上"
    )  # (T, 26)
    for action in action_seq:
        client.set_joint_positions({...})
```

5.8 第八步：放入周转盒

VLA技能：`VLA_PLACE`

5.9 第九步：查看结果

VLM任务：`VLM_READ` (非动作，纯视觉理解)

```
async def step9_check_result():
    rgb_screen = capture_screen_image()
    result = vlm_model.inference(
        image=rgb_screen,
        prompt="读取气密检测结果：PASS或FAIL"
    )
    return {"result": "PASS" if "PASS" in result else "FAIL"}
```

六、关键技术要点

5.1 力控精度

GR2 灵巧手需实现亚牛顿级力控（0.5-2N）避免损伤镜头和镜头帽。

实现方式：

```
# 通过调整电机PD参数实现柔顺控制
def set_force_control(client, hand, target_force):
    # 降低Kp增加柔顺性
    kp = max(10, 50 - target_force * 20)
    kd = 1.0
    client.set_motor_cfg(
        {f"{hand}_hand": [kp]*6},
        {f"{hand}_hand": [kd]*6}
    )
```

5.2 对准精度

镜头帽插拔需达到 **±0.3mm** 重复定位精度。

实现方式：

- 使用 RoboBrain VLA 模型进行视觉伺服
- 闭环反馈补偿位置误差
- 多次微调直到对准

```
async def fine_align_with_vla(target, tolerance=0.0003):
    while True:
        # 获取当前偏差
        offset = await P1_visual_locate(target)
        error = np.linalg.norm(offset["position"][:2])

        if error < tolerance:
            break

        # 微调补偿
        await M1_move_to_point(arm, compensated_pos, duration=0.5)
```

5.3 双手协调

M2 技能需支持主从手同步，时间差 **<100ms**。

实现方式：

```
async def bimanual_move(left_pos, right_pos, duration=2.0):
    client = AuroraClient.get_instance(domain_id=123)

    # 同时发送双臂指令
    for step in range(total_steps):
        client.set_joint_positions({
            "left_manipulator": interpolate(left_init, left_pos, step),
            "right_manipulator": interpolate(right_init, right_pos, step)
        })
        await asyncio.sleep(0.01)  # 100Hz控制频率
```

5.4 视觉引导

所有抓取操作需融合 P1 实时视觉伺服，补偿位置误差。

RoboBrain VLA 调用：

```
from inference import UnifiedInference

model = UnifiedInference("BAAI/RoboBrain2.0-7B")

# 视觉定位
result = model.inference(
    prompt="定位图中的摄像头产品",
    image=camera_image,
    task="grounding",
    enable_thinking=True
)

# 外观检测
result = model.inference(
    prompt="检测镜头是否有划痕或损伤",
    image=lens_image,
    task="general",
    enable_thinking=True
)
```

5.5 防碰撞

M4 轨迹规划需考虑工装、周转盒、检测台等环境障碍。

环境建模：

```
OBSTACLES = [
    {"name": "气密检测台", "bbox": [0.3, 0, 0.2, 0.5, 0.3, 0.4]},
    {"name": "周转盒_左", "bbox": [-0.3, 0.2, 0, 0.2, 0.3, 0.15]},
    {"name": "周转盒_右", "bbox": [0.5, 0.2, 0, 0.2, 0.3, 0.15]},
    {"name": "显示屏", "bbox": [-0.5, 0, 0.3, 0.3, 0.02, 0.4]},
]
```

六、部署指南

6.1 环境准备

```
# 1. 激活环境
conda activate robobrain2

# 2. 启动 RoboBrain VLA 模型服务
cd /home/phl/workspace/fmc3-robotics/projects/RoboBrain2.0
bash startup.sh
# 或手动启动:
# python inference.py --serve --host 0.0.0.0 --port 4567 --model-id /path/to/RoboBrain2.0-7B

# 3. 启动 Redis
redis-server &

# 4. 启动机器人仿真/实机
cd /home/phl/workspace/fourier_aurora_sdk
sudo bash docker_run.bash
# 容器内: python3 sim/start_simulate_root.py
```

6.2 启动技能服务

```
# 启动海康任务技能服务
cd /home/phl/workspace/fmc3-robotics/projects/RoboSkill/fmc3-robotics/fourier/gr2
python hikvision_skills.py
# 服务运行在 http://0.0.0.0:8001
```

6.3 启动 RoboOS

```
# 终端1: Master Agent
cd /home/phl/workspace/fmc3-robotics/projects/RoboOS/master
python run.py

# 终端2: Slaver Agent
cd /home/phl/workspace/fmc3-robotics/projects/RoboOS/slaver
python run.py

# 终端3: Web UI (可选)
cd /home/phl/workspace/fmc3-robotics/projects/RoboOS/deploy
python run.py
# 访问 http://127.0.0.1:8888
```

6.4 发送任务

```
import requests

# 发送完整任务
requests.post("http://localhost:5000/publish_task", json={
    "task": "执行海康摄像头气密检测流程"
})

# 或发送单步任务
requests.post("http://localhost:5000/publish_task", json={
    "task": "从周转盒中取出一个摄像头"
})
```

七、配置文件

7.1 Slaver 配置 (slaver/config.yaml)

```
# 机器人连接配置
robot:
  type: fourier_gr2
  domain_id: 123
  robot_name: gr2

# MCP技能服务
skills:
  - name: hikvision_inspection
    url: http://localhost:8001
    transport: streamable-http

# 工具调用配置
tool_call:
  enabled: true
  max_retries: 3
```

7.2 Master 配置 (master/config.yaml)

```yaml
# VLA模型服务
cloud_server:
  url: http://localhost:4567
  model: RoboBrain2.0-7B

# Redis配置
redis:
  host: localhost
  port: 6379

# 任务分解配置
task_decomposition:
  max_subtasks: 20
  enable_thinking: true
```

八、调试与测试

8.1 单技能测试

```python
# 测试单个技能
import asyncio
from hikvision_skills import P1_visual_locate, G2_pinch_grasp

async def test():
    # 测试视觉定位
    result = await P1_visual_locate("摄像头")
    print(f"定位结果: {result}")

    # 测试抓取
    result = await G2_pinch_grasp("left", force=1.0)
    print(f"抓取结果: {result}")

asyncio.run(test())
```

8.2 MCP 技能调试

```bash
# 使用 MCP 开发工具
cd /home/phl/workspace/fmc3-robotics/projects/RoboSkill/fmc3-robotics/fourier/gr2
mcp dev hikvision_skills.py
```

8.3 完整流程测试

```
# 仿真环境测试
python test_hikvision_flow.py --simulation

# 实机测试 (谨慎!)
python test_hikvision_flow.py --real --speed 0.5
```

九、故障处理

故障现象	可能原因	处理方法
抓取失败	定位偏差过大	检查视觉标定，重新定位
镜头帽盖不上	对准精度不足	增加视觉伺服迭代次数
双按钮无响应	同步时间差过大	检查网络延迟，调整同步策略
力控不稳定	Kp/Kd参数不当	重新调参，降低控制频率
轨迹碰撞	障碍物建模不准	更新环境模型，增加安全边界

十、基于VLA的原子技能实现方案

本章详细描述如何使用 **RoboBrain 2.0 VLA模型** 统一实现感知、移动、抓取、操作等原子技能。

10.1 VLA驱动的技能架构

```
┌──────────────────────────────────────────────────────────────────┐
│                      RoboBrain 2.0 VLA 模型                      │
│                       (7B / 32B 参数)                            │
├──────────────────────────────────────────────────────────────────┤
│  支持任务类型:                                                    │
│  • general     - 通用视觉问答、场景理解                          │
│  • grounding   - 视觉定位 (返回 bbox)                            │
│  • affordance  - 可操作区域预测 (抓取点)                         │
│  • trajectory  - 轨迹预测 (末端执行器路径)                       │
│  • pointing    - 多点定位 (空位检测)                             │
└──────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│                          VLA技能适配层                           │
│            将VLA输出转换为机器人控制指令                          │
├──────────────────────────────────────────────────────────────────┤
│  • 像素坐标 → 3D世界坐标 (相机标定)                              │
│  • 轨迹点 → 关节角度 (逆运动学)                                  │
│  • bbox → 抓取姿态 (抓取规划)                                    │
└──────────────────────────────────────────────────────────────────┘
```

```
                              |
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                    Fourier GR2 机器人控制                          │
│                    (Aurora Client API)                            │
└──────────────────────────────────────────────────────────────────┘
```

10.2 VLA模型能力与技能映射

原子技能	VLA任务类型	输入	输出	说明
P1 视觉定位	`grounding`	图像 + "定位摄像头"	bbox [x1,y1,x2,y2]	目标物体边界框
P2 外观检测	`general`	图像 + "检测镜头缺陷"	文本描述	缺陷分析与判断
P3 二维码扫描	`grounding`	图像 + "定位二维码"	bbox	配合扫码器使用
P4 屏幕读取	`general`	图像 + "读取检测结果"	PASS/FAIL	OCR+理解
M1 移动到点	`affordance`	图像 + "抓取摄像头"	抓取点坐标	确定目标点
M3 姿态调整	`general`	图像 + "如何调整使镜头朝上"	旋转指令	姿态规划
M4 轨迹规划	`trajectory`	图像 + "移动到香蕉"	轨迹点列表	避障路径
O4 精对准	`pointing`	图像 + "定位工装槽位中心"	点坐标	高精度定位
G2 抓取点	`affordance`	图像 + "抓取杯子把手"	affordance区域	最佳抓取位置

10.3 VLA驱动的技能实现代码

```python
# hikvision_vla_skills.py - 基于VLA的海康任务原子技能库

import asyncio
import numpy as np
from mcp.server.fastmcp import FastMCP
from fourier_aurora_client import AuroraClient
from inference import UnifiedInference

mcp = FastMCP("hikvision_vla", stateless_http=True, host="0.0.0.0", port=8001)

# 初始化VLA模型
vla_model = UnifiedInference("BAAI/RoboBrain2.0-7B")

# 相机内参 (需根据实际标定)
CAMERA_INTRINSICS = {
    "fx": 600.0, "fy": 600.0,
    "cx": 320.0, "cy": 240.0,
    "depth_scale": 0.001
}

# ==================== 坐标转换工具 ====================
```

```python
def pixel_to_world(pixel_x, pixel_y, depth, camera_pose):
    """像素坐标转世界坐标"""
    fx, fy = CAMERA_INTRINSICS["fx"], CAMERA_INTRINSICS["fy"]
    cx, cy = CAMERA_INTRINSICS["cx"], CAMERA_INTRINSICS["cy"]

    # 相机坐标系
    z = depth * CAMERA_INTRINSICS["depth_scale"]
    x = (pixel_x - cx) * z / fx
    y = (pixel_y - cy) * z / fy

    # 转换到世界坐标系 (简化, 实际需要相机外参)
    world_pos = np.array([x, y, z]) + camera_pose[:3]
    return world_pos.tolist()

def bbox_to_center(bbox):
    """bbox中心点"""
    x1, y1, x2, y2 = bbox
    return ((x1 + x2) / 2, (y1 + y2) / 2)

# ==================== 感知技能 (P) - VLA驱动 ====================

@mcp.tool()
async def P1_visual_locate_vla(target: str, image_path: str) -> dict:
    """【VLA】视觉定位目标物体

    使用RoboBrain VLA的grounding能力定位目标

    Args:
        target: 目标物体描述 (如"摄像头"、"镜头帽"、"绿色按钮")
        image_path: 当前相机图像路径

    Returns:
        {
            "bbox": [x1, y1, x2, y2],       # 像素坐标边界框
            "center": [cx, cy],              # 中心点像素坐标
            "world_pos": [x, y, z],          # 世界坐标 (需深度)
            "confidence": 0.95,
            "thinking": "..."                # VLA推理过程
        }
    """
    # 调用VLA grounding任务
    result = vla_model.inference(
        prompt=f"定位图中的{target}",
        image=image_path,
        task="grounding",
        enable_thinking=True,
        do_sample=False
    )

    # 解析bbox
    bbox_str = result.get("answer", "[0,0,0,0]")
    bbox = eval(bbox_str)  # [x1, y1, x2, y2]
```

```
    center = bbox_to_center(bbox)

    # TODO: 获取深度图，转换为世界坐标
    # depth = get_depth_at(center[0], center[1])
    # world_pos = pixel_to_world(center[0], center[1], depth, camera_pose)

    return {
        "bbox": bbox,
        "center": list(center),
        "world_pos": [0.0, 0.0, 0.0],  # 需要深度相机
        "confidence": 0.95,
        "thinking": result.get("thinking", "")
    }


@mcp.tool()
async def P2_visual_inspect_vla(target: str, image_path: str) -> dict:
    """【VLA】视觉检测外观缺陷

    使用RoboBrain VLA的通用理解能力检测缺陷

    Args:
        target: 检测目标 (如"镜头")
        image_path: 当前相机图像路径

    Returns:
        {
            "status": "normal" | "defect",
            "defects": ["划痕", "污渍"],
            "description": "详细描述",
            "thinking": "..."
        }
    """
    result = vla_model.inference(
        prompt=f"""仔细检查图中的{target}，判断是否存在以下缺陷:
1. 划痕或刮伤
2. 污渍或指纹
3. 破损或裂纹
4. 异物附着

请给出检测结论: 正常(normal)或有缺陷(defect)，并列出发现的问题。""",
        image=image_path,
        task="general",
        enable_thinking=True,
        do_sample=False
    )

    answer = result.get("answer", "").lower()
    status = "defect" if "defect" in answer or "缺陷" in answer else "normal"

    return {
        "status": status,
        "defects": [],  # 从answer中解析
```

```
        "description": result.get("answer", ""),
        "thinking": result.get("thinking", "")
    }


@mcp.tool()
async def P4_read_screen_vla(image_path: str) -> dict:
    """【VLA】读取屏幕气密检测结果

    Args:
        image_path: 屏幕图像路径

    Returns:
        {
            "result": "PASS" | "FAIL",
            "details": {"pressure": 0.5, "leak_rate": 0.001},
            "raw_text": "原始识别文本"
        }
    """
    result = vla_model.inference(
        prompt="""读取屏幕上显示的气密检测结果，提取以下信息:
1. 检测结果: PASS 或 FAIL
2. 压力值（如有）
3. 泄漏率（如有）
请以JSON格式返回。""",
        image=image_path,
        task="general",
        enable_thinking=True,
        do_sample=False
    )

    answer = result.get("answer", "").upper()
    test_result = "PASS" if "PASS" in answer else "FAIL"

    return {
        "result": test_result,
        "details": {},
        "raw_text": result.get("answer", ""),
        "thinking": result.get("thinking", "")
    }


# ==================== 移动技能 (M) - VLA驱动 ====================

@mcp.tool()
async def M4_plan_trajectory_vla(target: str, image_path: str) -> dict:
    """【VLA】轨迹规划 - 使用VLA预测末端执行器路径

    Args:
        target: 目标描述 (如"移动到摄像头位置")
        image_path: 当前场景图像

    Returns:
```

```
        {
            "trajectory": [(x1,y1), (x2,y2), ...],  # 像素坐标轨迹点
            "world_trajectory": [[x,y,z], ...],      # 世界坐标轨迹
            "thinking": "..."
        }
    """
    result = vla_model.inference(
        prompt=f"规划末端执行器从当前位置{target}的运动轨迹",
        image=image_path,
        task="trajectory",
        enable_thinking=True,
        do_sample=False
    )

    # 解析轨迹点
    trajectory_str = result.get("answer", "[]")
    trajectory = eval(trajectory_str)  # [(x1,y1), (x2,y2), ...]

    return {
        "trajectory": trajectory,
        "world_trajectory": [],  # 需要深度转换
        "num_points": len(trajectory),
        "thinking": result.get("thinking", "")
    }


# ==================== 抓取技能 (G) - VLA驱动 ====================

@mcp.tool()
async def G2_get_grasp_point_vla(target: str, image_path: str) -> dict:
    """【VLA】获取最佳抓取点 - 使用affordance预测

    Args:
        target: 抓取目标和方式 (如"双指抓取摄像头侧面")
        image_path: 当前场景图像

    Returns:
        {
            "grasp_bbox": [x1, y1, x2, y2],  # 抓取区域
            "grasp_center": [cx, cy],         # 抓取中心点
            "grasp_type": "pinch" | "power",  # 抓取类型
            "thinking": "..."
        }
    """
    result = vla_model.inference(
        prompt=f"确定{target}的最佳抓取区域",
        image=image_path,
        task="affordance",
        enable_thinking=True,
        do_sample=False
    )

    bbox_str = result.get("answer", "[0,0,0,0]")
```

```
    bbox = eval(bbox_str)
    center = bbox_to_center(bbox)

    return {
        "grasp_bbox": bbox,
        "grasp_center": list(center),
        "grasp_type": "pinch",  # 根据目标推断
        "thinking": result.get("thinking", "")
    }


# ==================== 操作技能 (O) - VLA驱动 ====================

@mcp.tool()
async def O4_fine_align_vla(target: str, image_path: str) -> dict:
    """【VLA】精对准 - 使用pointing获取精确位置

    Args:
        target: 对准目标 (如"工装槽位中心"、"镜头帽插入位置")
        image_path: 当前场景图像

    Returns:
        {
            "target_points": [(x1,y1), (x2,y2), ...],  # 目标点
            "primary_point": [x, y],                    # 主要对准点
            "thinking": "..."
        }
    """
    result = vla_model.inference(
        prompt=f"精确定位{target}的中心位置, 返回多个候选点",
        image=image_path,
        task="pointing",
        enable_thinking=True,
        do_sample=False
    )

    points_str = result.get("answer", "[]")
    points = eval(points_str)

    # 取平均作为主要对准点
    if points:
        primary = [
            sum(p[0] for p in points) / len(points),
            sum(p[1] for p in points) / len(points)
        ]
    else:
        primary = [0, 0]

    return {
        "target_points": points,
        "primary_point": primary,
        "thinking": result.get("thinking", "")
    }
```

```python
# ==================== 复合VLA技能 ====================

@mcp.tool()
async def vla_pick_object(target: str, image_path: str, arm: str = "left") -> dict:
    """【VLA复合技能】智能抓取物体

    组合使用 grounding + affordance + trajectory 完成抓取

    Args:
        target: 抓取目标 (如"摄像头")
        image_path: 当前场景图像
        arm: 使用哪只手臂 ("left" | "right")

    Returns:
        执行结果
    """
    client = AuroraClient.get_instance(domain_id=123)

    # Step 1: 定位目标
    locate_result = await P1_visual_locate_vla(target, image_path)
    print(f"[VLA] 定位结果: {locate_result['bbox']}")

    # Step 2: 获取抓取点
    grasp_result = await G2_get_grasp_point_vla(f"抓取{target}", image_path)
    print(f"[VLA] 抓取点: {grasp_result['grasp_center']}")

    # Step 3: 规划轨迹
    traj_result = await M4_plan_trajectory_vla(f"移动到{target}", image_path)
    print(f"[VLA] 轨迹点数: {traj_result['num_points']}")

    # Step 4: 执行抓取 (需要坐标转换)
    # TODO: 将像素坐标转换为关节角度并执行

    return {
        "success": True,
        "target": target,
        "grasp_point": grasp_result["grasp_center"],
        "trajectory_points": traj_result["num_points"],
        "vla_reasoning": {
            "locate": locate_result["thinking"][:200],
            "grasp": grasp_result["thinking"][:200],
            "trajectory": traj_result["thinking"][:200]
        }
    }


@mcp.tool()
async def vla_place_object(target_location: str, image_path: str, arm: str = "right") ->
dict:
    """【VLA复合技能】智能放置物体
```

```
    组合使用 pointing + trajectory 完成放置

    Args:
        target_location: 放置位置描述 (如"工装槽位"、"周转盒空位")
        image_path: 当前场景图像
        arm: 使用哪只手臂

    Returns:
        执行结果
    """
    # Step 1: 精确定位放置点
    align_result = await O4_fine_align_vla(target_location, image_path)
    print(f"[VLA] 放置点: {align_result['primary_point']}")

    # Step 2: 规划放置轨迹
    traj_result = await M4_plan_trajectory_vla(f"移动到{target_location}上方", image_path)

    # Step 3: 执行放置
    # TODO: 执行实际放置动作

    return {
        "success": True,
        "target_location": target_location,
        "place_point": align_result["primary_point"],
        "vla_reasoning": align_result["thinking"][:300]
    }


# ==================== 完整任务流程 (VLA版本) ====================

@mcp.tool()
async def vla_hikvision_step1_pick_camera(image_path: str) -> dict:
    """【VLA】第一步: 从周转盒取摄像头

    完整流程: 定位 → 规划轨迹 → 获取抓取点 → 执行抓取 → 抬起
    """
    return await vla_pick_object("周转盒中的摄像头", image_path, arm="left")


@mcp.tool()
async def vla_hikvision_step2_remove_cap(image_path: str) -> dict:
    """【VLA】第二步: 摘除镜头帽并检查

    流程: 定位镜头帽 → 抓取 → 拔出 → 放置 → 检查镜头
    """
    # 1. 定位镜头帽
    cap_result = await P1_visual_locate_vla("摄像头上的镜头帽", image_path)

    # 2. 获取镜头帽抓取点
    grasp_result = await G2_get_grasp_point_vla("捏住镜头帽边缘", image_path)

    # 3. 定位放置区域
    place_result = await O4_fine_align_vla("气密检测台左侧空位", image_path)
```

```python
    # 4. 检查镜头外观
    inspect_result = await P2_visual_inspect_vla("摄像头镜头", image_path)

    return {
        "cap_location": cap_result["center"],
        "grasp_point": grasp_result["grasp_center"],
        "place_point": place_result["primary_point"],
        "inspection": inspect_result["status"],
        "inspection_detail": inspect_result["description"]
    }


@mcp.tool()
async def vla_hikvision_step4_place_fixture(image_path: str) -> dict:
    """【VLA】第四步：放入气密检测工装

    流程：定位工装槽位 → 精对准 → 规划轨迹 → 放置
    """
    # 1. 定位工装槽位
    fixture_result = await P1_visual_locate_vla("气密检测工装的正方形槽位", image_path)

    # 2. 精确对准槽位中心
    align_result = await O4_fine_align_vla("工装槽位的精确中心", image_path)

    # 3. 规划放置轨迹
    traj_result = await M4_plan_trajectory_vla("移动到工装槽位正上方", image_path)

    return {
        "fixture_bbox": fixture_result["bbox"],
        "align_point": align_result["primary_point"],
        "trajectory": traj_result["trajectory"],
        "thinking": fixture_result["thinking"]
    }


@mcp.tool()
async def vla_hikvision_step5_press_buttons(image_path: str) -> dict:
    """【VLA】第五步：按下启动按钮

    流程：定位两个绿色按钮 → 规划双臂轨迹 → 同步按压
    """
    # 1. 定位左侧绿色按钮
    left_btn = await P1_visual_locate_vla("左侧绿色启动按钮", image_path)

    # 2. 定位中间绿色按钮
    mid_btn = await P1_visual_locate_vla("中间绿色启动按钮", image_path)

    return {
        "left_button": left_btn["center"],
        "middle_button": mid_btn["center"],
        "left_thinking": left_btn["thinking"][:200],
        "middle_thinking": mid_btn["thinking"][:200]
```

```
    }


@mcp.tool()
async def vla_hikvision_step7_replace_cap(image_path: str) -> dict:
    """【VLA】第七步：盖上镜头帽

    流程：定位镜头帽 → 定位镜头 → 精对准 → 插入
    """
    # 1. 定位之前放置的镜头帽
    cap_result = await P1_visual_locate_vla("放置在台面上的镜头帽", image_path)

    # 2. 定位右手中摄像头的镜头位置
    lens_result = await P1_visual_locate_vla("摄像头镜头开口", image_path)

    # 3. 精确对准 (关键步骤，需要±0.3mm精度)
    align_result = await O4_fine_align_vla("镜头帽与镜头的对准位置", image_path)

    return {
        "cap_location": cap_result["center"],
        "lens_location": lens_result["center"],
        "align_points": align_result["target_points"],
        "primary_align": align_result["primary_point"],
        "thinking": align_result["thinking"]
    }


@mcp.tool()
async def vla_hikvision_step8_place_box(image_path: str, current_index: int = 0) -> dict:
    """【VLA】第八步：放入周转盒

    流程：定位下一个空位 → 规划轨迹 → 放置

    Args:
        current_index: 当前放置序号 (从0开始，从右到左、从上到下)
    """
    # 1. 定位周转盒中的空位
    slot_result = await O4_fine_align_vla(
        f"周转盒中第{current_index + 1}个空位（从右到左、从上到下顺序）",
        image_path
    )

    # 2. 规划放置轨迹
    traj_result = await M4_plan_trajectory_vla("移动到周转盒空位上方", image_path)

    return {
        "slot_index": current_index,
        "slot_position": slot_result["primary_point"],
        "trajectory": traj_result["trajectory"],
        "next_index": current_index + 1
    }
```

```python
@mcp.tool()
async def vla_hikvision_step9_check_result(image_path: str) -> dict:
    """【VLA】第九步：查看检测结果

    流程：定位屏幕 → OCR读取结果 → 判断PASS/FAIL
    """
    # 1. 定位显示屏
    screen_result = await P1_visual_locate_vla("气密检测结果显示屏", image_path)

    # 2. 读取检测结果
    read_result = await P4_read_screen_vla(image_path)

    return {
        "screen_location": screen_result["center"],
        "test_result": read_result["result"],
        "details": read_result["details"],
        "raw_text": read_result["raw_text"],
        "passed": read_result["result"] == "PASS"
    }


if __name__ == "__main__":
    print("🚀 Starting Hikvision VLA Skill Server on port 8001...")
    print("📦 VLA Model: RoboBrain2.0-7B")
    print("🤖 Robot: Fourier GR2")
    mcp.run(transport="streamable-http")
```

10.4 VLA技能与传统技能对比

对比项	传统技能 (硬编码)	VLA驱动技能
定位方式	预设坐标 / 标记检测	语义理解 + grounding
抓取规划	固定抓取姿态	affordance预测最佳抓取点
轨迹规划	RRT/PRM算法	VLA trajectory预测
缺陷检测	传统CV算法	VLM推理 + 思维链
泛化能力	差，需要重新编程	强，zero-shot迁移
推理速度	快 (<10ms)	较慢 (100-500ms)
精度	高 (标定后)	中等 (需视觉伺服)

10.5 VLA视觉伺服闭环

对于高精度操作（如镜头帽插入±0.3mm），需要VLA视觉伺服闭环：

```python
async def vla_visual_servo(target: str, tolerance: float = 0.0003):
    """VLA视觉伺服闭环控制

    通过多次VLA推理，逐步逼近目标位置
```

```
    """
    client = AuroraClient.get_instance(domain_id=123)
    max_iterations = 10

    for i in range(max_iterations):
        # 1. 拍摄当前图像
        image = capture_current_image()

        # 2. VLA获取当前偏差
        align_result = await O4_fine_align_vla(target, image)
        current_point = align_result["primary_point"]

        # 3. 计算像素偏差 (假设目标在图像中心)
        image_center = [320, 240]  # 640x480图像
        pixel_error = np.linalg.norm(
            np.array(current_point) - np.array(image_center)
        )

        # 4. 转换为世界坐标偏差 (需要相机标定)
        # 假设1像素 ≈ 0.1mm (需要实际标定)
        world_error = pixel_error * 0.0001  # 米

        print(f"[VLA Servo] 迭代 {i+1}: 像素偏差={pixel_error:.1f}, 世界偏差=
{world_error*1000:.2f}mm")

        if world_error < tolerance:
            print(f"[VLA Servo] ✓ 对准完成, 精度: {world_error*1000:.3f}mm")
            return {"success": True, "final_error": world_error}

        # 5. 计算补偿量并移动
        compensation = [
            -(current_point[0] - image_center[0]) * 0.0001,
            -(current_point[1] - image_center[1]) * 0.0001,
            0  # Z方向不变
        ]

        # 6. 发送微调指令
        await move_end_effector_delta(client, compensation)
        await asyncio.sleep(0.2)  # 等待稳定

    return {"success": False, "final_error": world_error}
```

10.6 VLA + 传统控制混合架构

推荐采用 **VLA感知 + 传统控制** 的混合架构，发挥各自优势：

```
┌──────────────────────────────────────────────────────────┐
│                      高层任务规划                          │
│                   RoboBrain VLA (7B)                      │
│          • 场景理解 • 任务分解 • 异常处理                  │
└──────────────────────────┬──────────────────────────────┘
                           │
```

```
┌──────────────────────────▼──────────────────────────┐
│                    中层感知层                          │
│                  RoboBrain VLA                        │
│    • grounding定位 • affordance抓取点 • trajectory规划  │
└──────────────────────────┬──────────────────────────┘
                           │ 目标位置/轨迹点
┌──────────────────────────▼──────────────────────────┐
│                    底层控制层                          │
│               传统机器人控制 (100Hz)                    │
│    • 逆运动学 • 力控 • 轨迹插值 • 安全限位              │
└──────────────────────────────────────────────────────┘
```

分工建议:

层级	使用VLA	使用传统方法
任务理解	✓ 自然语言理解	
目标定位	✓ grounding	
抓取规划	✓ affordance	
路径规划	✓ trajectory (粗规划)	✓ 精细插值
轨迹执行		✓ 100Hz控制
力控制		✓ PD控制
精对准	✓ pointing + 伺服	✓ 微调控制

10.7 VLA部署配置 (RTX 4090 48GB 魔改版)

基于 **RTX 4090 48GB 魔改显存** 的单卡部署方案，可以运行完整的 RoboBrain 2.0-7B 或量化的 32B 模型。

硬件配置

```yaml
# 硬件规格
gpu:
  model: "RTX 4090 48GB (魔改)"
  vram: 48GB
  cuda_cores: 16384
  tensor_cores: 512
  memory_bandwidth: 1008 GB/s

# 可部署的模型组合
deployment_options:
  # 方案1: 单模型全精度
  option_1:
    model: "RoboBrain2.0-7B"
    precision: "FP16"
    vram_usage: "~18GB"
    inference_speed: "~50 tokens/s"
```

```yaml
  # 方案2: 7B + VLA Controller 同时运行
  option_2:
    robobrain: "RoboBrain2.0-7B (FP16)"
    vla_controller: "DexGraspVLA Controller"
    vram_usage: "~25GB"

  # 方案3: 32B 4bit量化
  option_3:
    model: "RoboBrain2.0-32B"
    precision: "INT4 (AWQ/GPTQ)"
    vram_usage: "~20GB"
    inference_speed: "~25 tokens/s"

  # 方案4: 7B全精度 + SAM + 视频分割
  option_4:
    robobrain: "RoboBrain2.0-7B (FP16)"
    sam: "SAM-ViT-H"
    cutie: "Cutie (视频分割)"
    vram_usage: "~30GB"
```

推荐部署配置

```yaml
# config/vla_config_4090_48gb.yaml

# ============ RTX 4090 48GB 专用配置 ============

# RoboBrain VLA 模型 (任务分解 + 视觉理解)
robobrain:
  model_id: "BAAI/RoboBrain2.0-7B"
  device: "cuda:0"
  precision: "float16"           # 48GB可以跑全精度FP16
  load_in_4bit: false            # 无需4bit量化
  load_in_8bit: false            # 无需8bit量化
  torch_dtype: "float16"
  max_memory: "40GB"             # 预留8GB给其他模型

# DexGraspVLA Controller (动作生成)
vla_controller:
  checkpoint: "/path/to/dexgraspvla-controller"
  device: "cuda:0"
  precision: "float16"
  vram_reserved: "6GB"

# SAM 分割模型
sam:
  model_type: "vit_h"            # 最大模型, 48GB足够
  checkpoint: "/path/to/sam_vit_h.pth"
  device: "cuda:0"
  vram_usage: "~4GB"

# vLLM 服务配置 (RoboBrain)
vllm_server:
```

```yaml
  host: "0.0.0.0"
  port: 4567
  gpu_memory_utilization: 0.85   # 留15%给其他模型
  max_model_len: 16384           # 48GB可以支持更长上下文
  max_num_seqs: 8                # 并发请求数
  dtype: "float16"
  trust_remote_code: true
  enable_chunked_prefill: true

# 推理参数
inference:
  enable_thinking: true
  do_sample: false
  temperature: 0.1
  max_new_tokens: 2048           # 48GB可以输出更长

# 相机配置
camera:
  type: "realsense_d435"
  resolution: [640, 480]
  fps: 30
  depth_enabled: true
```

启动命令

```bash
# ============ RTX 4090 48GB 启动脚本 ============

# 1. 启动 RoboBrain VLA 服务 (FP16全精度)
vllm serve BAAI/RoboBrain2.0-7B \
    --host 0.0.0.0 \
    --port 4567 \
    --dtype float16 \
    --gpu-memory-utilization 0.7 \
    --max-model-len 16384 \
    --max-num-seqs 8 \
    --trust-remote-code \
    --enable-chunked-prefill

# 2. 或者运行32B量化版本 (AWQ 4bit)
vllm serve BAAI/RoboBrain2.0-32B \
    --host 0.0.0.0 \
    --port 4567 \
    --quantization awq \
    --dtype float16 \
    --gpu-memory-utilization 0.85 \
    --max-model-len 8192 \
    --trust-remote-code

# 3. 启动 VLA Controller (DexGraspVLA)
python -m controller.serve \
    --checkpoint /path/to/dexgraspvla-controller \
    --device cuda:0 \
```

```
    --port 8002

# 4. 启动海康任务技能服务
cd /home/phl/workspace/fmc3-robotics/projects/RoboSkill/fmc3-robotics/fourier/gr2
python hikvision_vla_skills.py
```

显存分配方案

```
┌──────────────────────────────────────────────────────────────────────┐
│                    RTX 4090 48GB 显存分配                            │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐      │
│   │         RoboBrain 2.0-7B (FP16)                          │      │
│   │                   ~18 GB                                 │      │
│   └──────────────────────────────────────────────────────────┘      │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐      │
│   │         DexGraspVLA Controller (Diffusion)               │      │
│   │                   ~6 GB                                  │      │
│   └──────────────────────────────────────────────────────────┘      │
│                                                                      │
│   ┌────────────────────────┐  ┌──────────────────────────┐         │
│   │   SAM-ViT-H            │  │   Cutie (视频分割)        │         │
│   │     ~4 GB              │  │     ~2 GB                 │         │
│   └────────────────────────┘  └──────────────────────────┘         │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐      │
│   │         KV Cache + 中间激活 + 预留                        |      |
│   │                   ~18 GB                                 │      │
│   └──────────────────────────────────────────────────────────┘      │
│                                                                      │
│  总计: 18 + 6 + 4 + 2 + 18 = 48 GB ✓                                |
└──────────────────────────────────────────────────────────────────────┘
```

32B模型量化部署方案

48GB显存可以运行量化的32B模型，获得更强的推理能力：

```
# 使用 AWQ 4bit 量化加载 32B 模型
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "BAAI/RoboBrain2.0-32B",
    device_map="cuda:0",
    torch_dtype=torch.float16,
    quantization_config={
        "load_in_4bit": True,
        "bnb_4bit_compute_dtype": torch.float16,
        "bnb_4bit_quant_type": "nf4",
        "bnb_4bit_use_double_quant": True,
    }
```

```
)
# 4bit量化后 32B 模型约占用 ~20GB 显存
```

性能预估

模型配置	显存占用	推理速度	适用场景
7B FP16	~18GB	~50 tok/s	实时任务执行
7B + VLA Controller	~25GB	~40 tok/s	完整技能执行
32B INT4 (AWQ)	~20GB	~25 tok/s	复杂任务分解
7B + SAM + Cutie	~30GB	~35 tok/s	视频理解任务

48GB vs 24GB 优势

特性	24GB (原版4090)	48GB (魔改)
7B全精度	✓ 需要量化	✓ FP16无损
32B模型	✗ 无法运行	✓ 4bit量化可运行
多模型并行	✗ 显存不足	✓ 可同时运行多个模型
长上下文	8K tokens	16K+ tokens
批量推理	batch=2	batch=8

10.8 性能优化建议

1. **模型量化**：使用4bit量化降低显存占用

```
python inference.py --serve --load-in-4bit --model-id RoboBrain2.0-7B
```

2. **批量推理**：同时处理多个VLA请求

```
results = vla_model.batch_inference([
    {"prompt": "定位摄像头", "image": img1, "task": "grounding"},
    {"prompt": "定位镜头帽", "image": img1, "task": "grounding"},
])
```

3. **结果缓存**：对于静态场景，缓存VLA结果

```
from functools import lru_cache

@lru_cache(maxsize=100)
def cached_locate(target: str, image_hash: str):
    return vla_model.inference(...)
```

4. **异步流水线**：VLA推理与机器人动作并行

```
# 机器人执行动作A的同时，VLA规划动作B
task_a = asyncio.create_task(execute_action(action_a))
task_b = asyncio.create_task(vla_plan_next(image))
await asyncio.gather(task_a, task_b)
```

十一、参考资料

- RoboBrain 2.0 文档
- RoboBrain 2.0 论文
- RoboOS 文档
- Fourier Aurora SDK API
- MCP 协议规范